

TOPIC-6: BACKTRACKING ALGORITHM

Q1. N-Queens Problem Visualization

Aim:

To solve and visualize the N-Queens problem using the Backtracking algorithm for different values of N.

Algorithm:

Step 1: Create an $N \times N$ chessboard.

Step 2: Start placing queens row by row.

Step 3: Check whether a queen can be safely placed in a column.

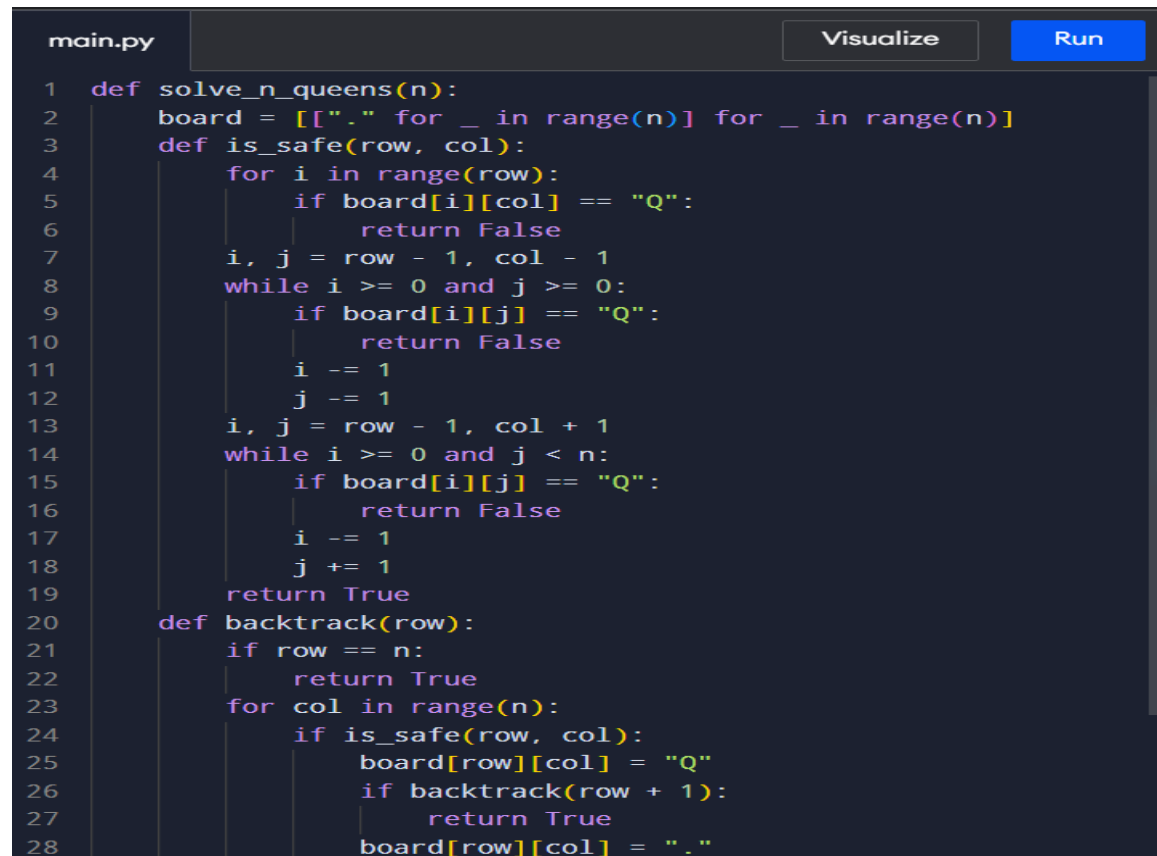
Step 4: Ensure that no two queens attack each other horizontally, vertically, or diagonally.

Step 5: If a safe position is found, place the queen.

Step 6: If no safe position exists, backtrack and change the previous queen's position.

Step 7: Print the valid chessboard solution.

Python Code:

A screenshot of a Python IDE window titled 'main.py'. The code implements the N-Queens problem using backtracking. It defines a 'solve_n_queens(n)' function that initializes an n x n board with '.' characters. A helper function 'is_safe(row, col)' checks if a queen can be placed at (row, col) by verifying no queens are in the same column or diagonals. The 'backtrack(row)' function recursively places queens row by row, returning True when a solution is found. The main function calls 'solve_n_queens(n)' and prints the resulting board.

```
main.py Visualize Run
1 def solve_n_queens(n):
2     board = [["." for _ in range(n)] for _ in range(n)]
3     def is_safe(row, col):
4         for i in range(row):
5             if board[i][col] == "Q":
6                 return False
7         i, j = row - 1, col - 1
8         while i >= 0 and j >= 0:
9             if board[i][j] == "Q":
10                return False
11            i -= 1
12            j -= 1
13        i, j = row - 1, col + 1
14        while i >= 0 and j < n:
15            if board[i][j] == "Q":
16                return False
17            i -= 1
18            j += 1
19        return True
20    def backtrack(row):
21        if row == n:
22            return True
23        for col in range(n):
24            if is_safe(row, col):
25                board[row][col] = "Q"
26                if backtrack(row + 1):
27                    return True
28                board[row][col] = "."
```

```

29         return False
30     backtrack(0)
31     return board
32 for n in [4, 5, 8]:
33     print(f"\nN = {n}")
34     solution = solve_n_queens(n)
35     for row in solution:
36         print(" ".join(row))

```

Output:

```

Output

N = 4
. Q . .
. . . Q
Q . . .
. . Q .

N = 5
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

N = 8
Q . . . . . . .
. . . . Q . . .
. . . . . . Q
. . . . Q . .
. . Q . . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . . .

=== Code Execution Successful ===

```

Time Complexity – $O(N!)$ | Space Complexity – $O(N^2)$

Q2. Generalized N-Queens Problem

Aim:

To solve generalized versions of the N-Queens problem with different board sizes and restricted positions using Backtracking.

Algorithm:

Step 1: Create the required board.

Step 2: Define obstacles or restricted positions.

Step 3: Place queens row by row.

Step 4: Check whether the position is safe.

Step 5: Ensure the position is not an obstacle or restricted location.

Step 6: If safe, place the queen and continue.

Step 7: If no solution is possible, backtrack.

Python Code:

```
main.py Visualize Run
1 def generalized_queens(rows, cols, obstacles=set()):
2     board = [["." for _ in range(cols)] for _ in range(rows)]
3     def is_safe(row, col):
4         if (row, col) in obstacles:
5             return False
6         for i in range(row):
7             for j in range(cols):
8                 if board[i][j] == "Q":
9                     if j == col or abs(i - row) == abs(j -
10                        col):
11                         return False
12             return True
13     def backtrack(row):
14         if row == rows:
15             return True
16         for col in range(cols):
17             if is_safe(row, col):
18                 board[row][col] = "Q"
19                 if backtrack(row + 1):
20                     return True
21                 board[row][col] = "."
22         return False
23     if backtrack(0):
24         return board
25     return None
26 obstacles = {(2, 2), (4, 4)}
27 solution = generalized_queens(5, 5, obstacles)
28 if solution:
```

```

28     for row in solution:
29         print(" ".join(row))
30 else:
31     print("No solution")

```

Output:

```

Output
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

```

Time Complexity – $O(N!)$ | Space Complexity – $O(N^2)$

Q3. Sudoku Solver Using Backtracking

Aim:

To solve a Sudoku puzzle by filling all empty cells using the Backtracking algorithm.

Algorithm:

Step 1: Find an empty cell represented by ..

Step 2: Try numbers from 1 to 9.

Step 3: Check whether the number is valid in:

- Row
- Column
- 3×3 sub-grid

Step 4: If valid, place the number.

Step 5: Recursively solve the next empty cell.

Step 6: If no solution is possible, remove the number and backtrack.

Step 7: Print the completed Sudoku board.

Python Code:

```

main.py Visualize Run
1 def solveSudoku(board):
2     def is_valid(row, col, num):
3         for x in range(9):
4             if board[row][x] == num:
5                 return False
6         for x in range(9):
7             if board[x][col] == num:
8                 return False
9         start_row = (row // 3) * 3
10        start_col = (col // 3) * 3
11        for i in range(3):
12            for j in range(3):
13                if board[start_row + i][start_col + j] == num:
14                    return False
15        return True
16    def solve():
17        for row in range(9):
18            for col in range(9):
19                if board[row][col] == ".":
20                    for num in "123456789":
21                        if is_valid(row, col, num):
22                            board[row][col] = num
23                            if solve():
24                                return True
25                            board[row][col] = "."
26        return True
27    solve()
28
29    return board
30 board = [
31     ["5","3",".", ".", ".", "7",".", ".", ".", "."],
32     ["6",".", ".", ".", "1","9","5",".", ".", "."],
33     [".","9","8",".", ".", ".", ".", "6","."],
34     ["8",".", ".", ".", "6",".", ".", ".", "3"],
35     ["4",".", ".", "8",".", "3",".", ".", "1"],
36     ["7",".", ".", ".", "2",".", ".", ".", "6"],
37     [".","6",".", ".", ".", ".", ".", "2","8","."],
38     [".",".", ".", "4","1","9",".", ".", ".", "5"],
39     [".",".", ".", ".", "8",".", ".", ".", "7","9"]
40 ]
41 solveSudoku(board)
42 for row in board:
43     print(row)--

```

Output:

['5', '3', '4', '6', '7', '8', '9', '1', '2']

['6', '7', '2', '1', '9', '5', '3', '4', '8']

['1', '9', '8', '3', '4', '2', '5', '6', '7']

['8', '5', '9', '7', '6', '1', '4', '2', '3']

['4', '2', '6', '8', '5', '3', '7', '9', '1']

['7', '1', '3', '9', '2', '4', '8', '5', '6']

['9', '6', '1', '5', '3', '7', '2', '8', '4']

['2', '8', '7', '4', '1', '9', '6', '3', '5']

['3', '4', '5', '2', '8', '6', '1', '7', '9']

Time Complexity – $O(9^M)$ | Space Complexity – $O(M)$

Q4. Sudoku Puzzle Using Backtracking

Aim:

To implement another Sudoku solver using Backtracking and fill all empty cells according to Sudoku rules.

Algorithm:

Step 1: Find an empty position.

Step 2: Try digits from 1 to 9.

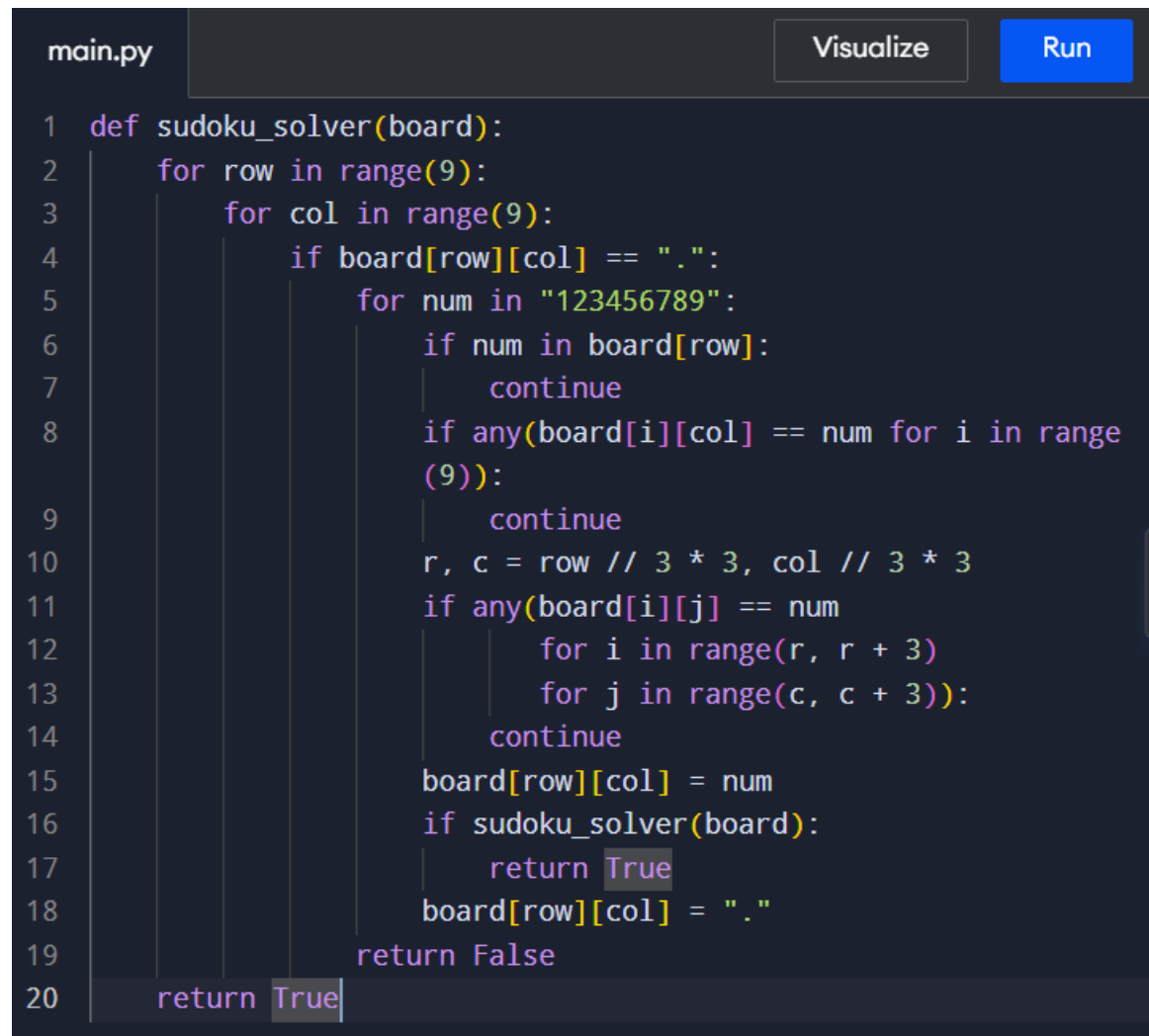
Step 3: Check row, column, and 3×3 box constraints.

Step 4: Place a valid digit.

Step 5: Recursively solve the remaining board.

Step 6: Backtrack if the selected digit leads to no solution.

Python Code:

A screenshot of a Python IDE window titled 'main.py'. The window has a dark background with light-colored text. On the right side, there are two buttons: 'Visualize' and 'Run'. The code is a recursive function 'sudoku_solver' that takes a 9x9 board as input. It iterates through rows and columns, checking for empty cells. For each empty cell, it tries digits from 1 to 9, skipping those already present in the row, column, or 3x3 box. If a valid digit is found, it places it and recursively calls the solver for the next state. If no solution is found, it backtracks by removing the digit and tries the next one. The function returns True if a solution is found and False otherwise.

```
1 def sudoku_solver(board):
2     for row in range(9):
3         for col in range(9):
4             if board[row][col] == ".":
5                 for num in "123456789":
6                     if num in board[row]:
7                         continue
8                     if any(board[i][col] == num for i in range
9                           (9)):
10                        continue
11                     r, c = row // 3 * 3, col // 3 * 3
12                     if any(board[i][j] == num
13                           for i in range(r, r + 3)
14                           for j in range(c, c + 3)):
15                         continue
16                     board[row][col] = num
17                     if sudoku_solver(board):
18                         return True
19                     board[row][col] = "."
20     return False
```

Time Complexity – $O(9^M)$ | Space Complexity – $O(M)$

Q5. Target Sum

Aim:

To find the number of different expressions formed using + and - signs that evaluate to the given target.

Algorithm:

Step 1: Start from the first number.

Step 2: Add the current number to the sum.

Step 3: Recursively process the next number.

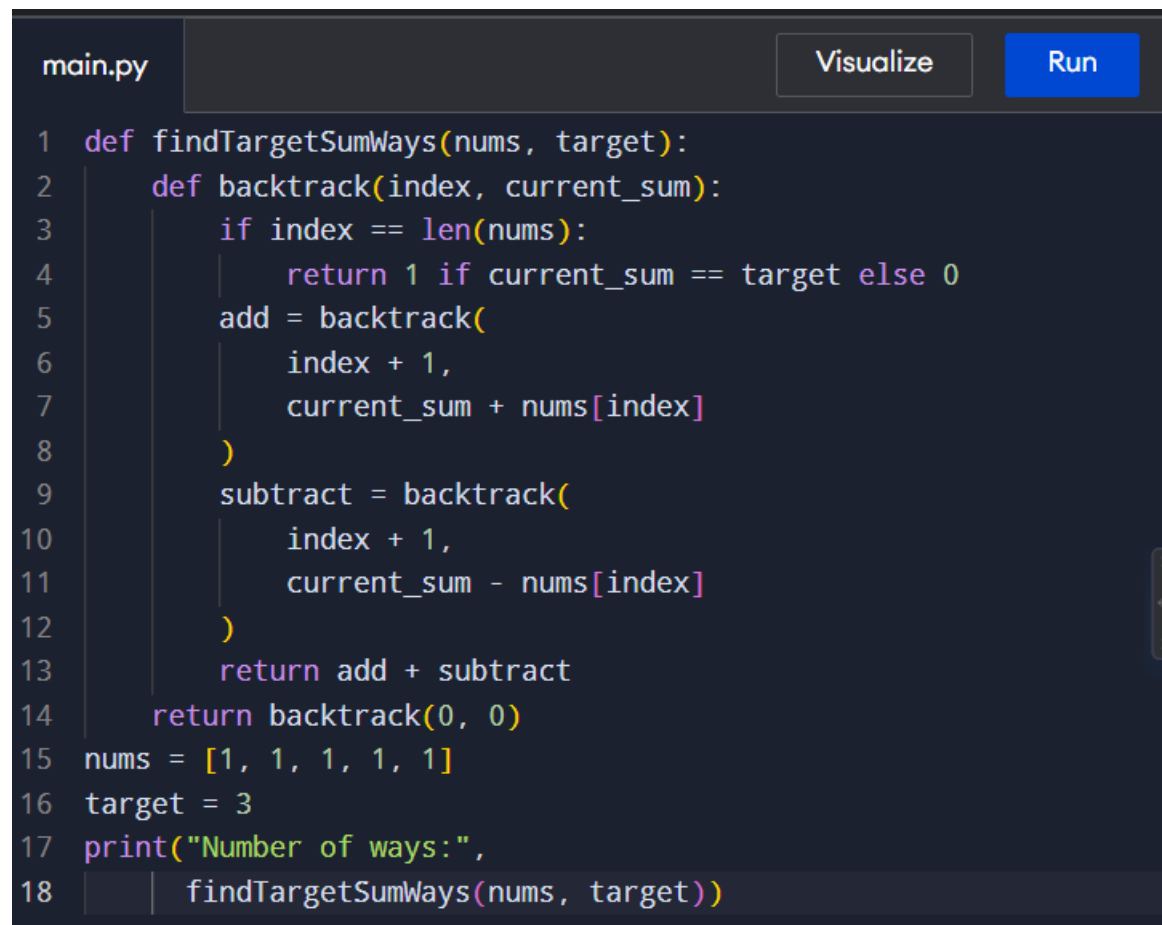
Step 4: Subtract the current number from the sum.

Step 5: Recursively process the next number.

Step 6: When all numbers are processed, check whether the sum equals the target.

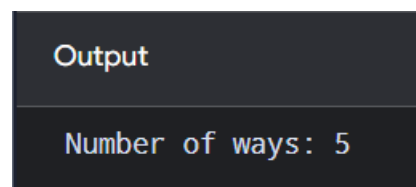
Step 7: Count all valid expressions.

Python Code:



```
main.py Visualize Run
1 def findTargetSumWays(nums, target):
2     def backtrack(index, current_sum):
3         if index == len(nums):
4             return 1 if current_sum == target else 0
5         add = backtrack(
6             index + 1,
7             current_sum + nums[index]
8         )
9         subtract = backtrack(
10            index + 1,
11            current_sum - nums[index]
12        )
13        return add + subtract
14    return backtrack(0, 0)
15 nums = [1, 1, 1, 1, 1]
16 target = 3
17 print("Number of ways:",
18       findTargetSumWays(nums, target))
```

Output:



```
Output
Number of ways: 5
```

Time Complexity – $O(2^N)$ | Space Complexity – $O(N)$

Q6. Sum of Subarray Minimums

Aim:

To find the sum of minimum values of all contiguous subarrays.

Algorithm:

Step 1: Generate every possible subarray.

Step 2: Find the minimum element of each subarray.

Step 3: Add the minimum value to the result.

Step 4: Repeat for all subarrays.

Step 5: Return the final sum modulo $10^9 + 7$.

Python Code:

```
main.py Visualize Run
1 def sumSubarrayMins(arr):
2     MOD = 10**9 + 7
3     result = 0
4     n = len(arr)
5     for i in range(n):
6         minimum = arr[i]
7         for j in range(i, n):
8             minimum = min(minimum, arr[j])
9             result += minimum
10    return result % MOD
11 arr = [3, 1, 2, 4]
12 print("Sum of subarray minimums:",
13       sumSubarrayMins(arr))
14 arr2 = [11, 81, 94, 43, 3]
15 print("Sum of subarray minimums:",
16       sumSubarrayMins(arr2))
```

Output:

```
Output
Sum of subarray minimums: 17
Sum of subarray minimums: 444
```

Time Complexity – $O(N^2)$ | Space Complexity – $O(N)$

Q7. Combination Sum

Aim:

To find all unique combinations where candidate numbers can be used unlimited times to obtain the target sum.

Algorithm:

Step 1: Sort the candidate numbers.

Step 2: Start with an empty combination.

Step 3: Select a candidate number.

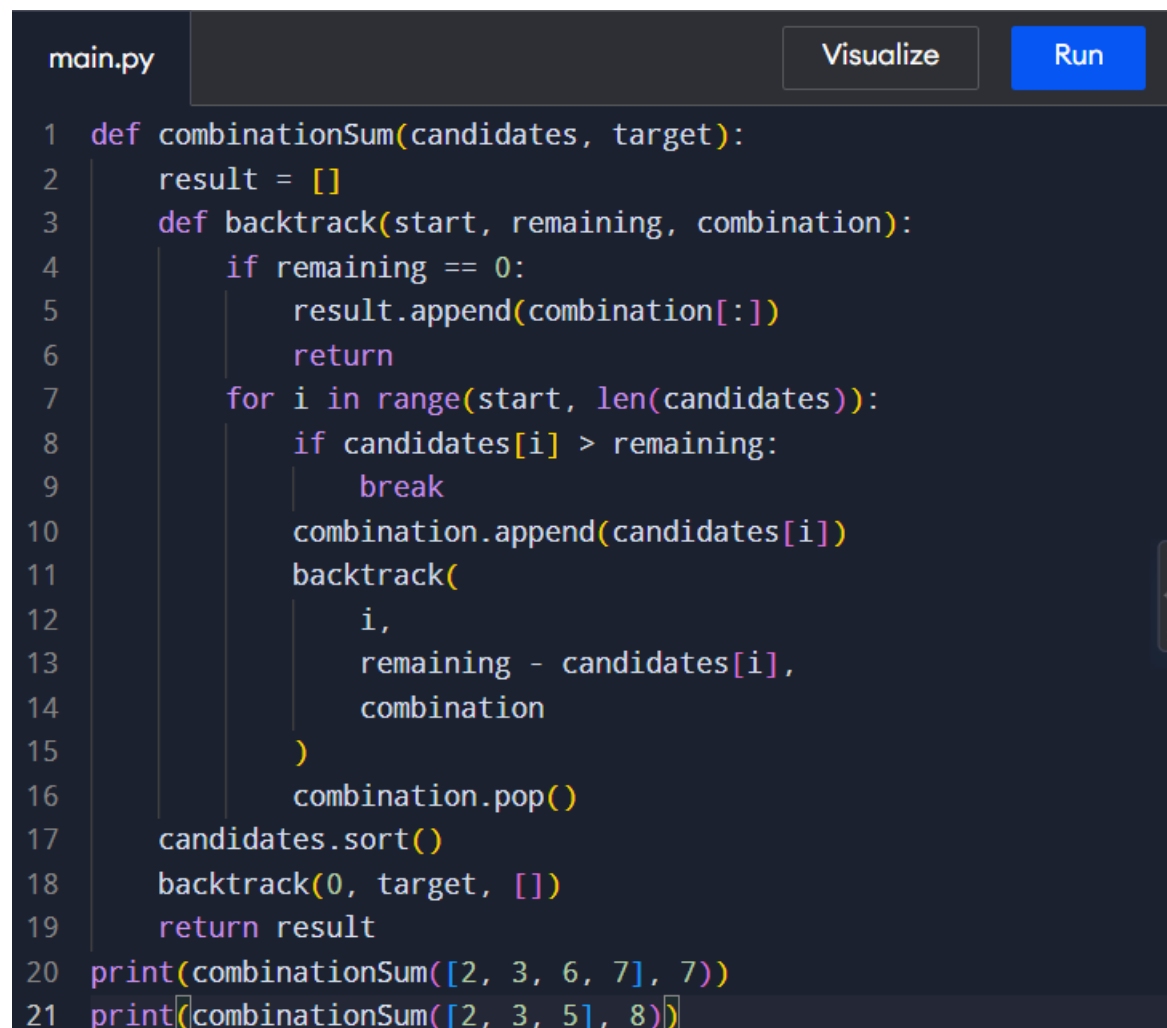
Step 4: Add it to the current combination.

Step 5: Recursively search for the remaining target.

Step 6: Allow the same number to be selected again.

Step 7: When the target becomes zero, store the combination.

Python Code:

A screenshot of a Python IDE with a dark theme. The file name 'main.py' is in the top left. There are two buttons in the top right: 'Visualize' and 'Run'. The code is as follows:

```
1 def combinationSum(candidates, target):
2     result = []
3     def backtrack(start, remaining, combination):
4         if remaining == 0:
5             result.append(combination[:])
6             return
7         for i in range(start, len(candidates)):
8             if candidates[i] > remaining:
9                 break
10            combination.append(candidates[i])
11            backtrack(
12                i,
13                remaining - candidates[i],
14                combination
15            )
16            combination.pop()
17    candidates.sort()
18    backtrack(0, target, [])
19    return result
20 print(combinationSum([2, 3, 6, 7], 7))
21 print(combinationSum([2, 3, 5], 8))
```

Output:

Output

```
[[2, 2, 3], [7]]  
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
```

Time Complexity – $O(2^N)$ | Space Complexity – $O(N)$

Q8. Combination Sum II

Aim:

To find all unique combinations where each candidate can be used only once.

Algorithm:

Step 1: Sort the candidate numbers.

Step 2: Start Backtracking.

Step 3: Select a number only once.

Step 4: Skip duplicate values.

Step 5: Add valid combinations whose sum equals the target.

Step 6: Return all unique combinations.

Python Code:

```
main.py Visualize Run  
1 def combinationSum2(candidates, target):  
2     candidates.sort()  
3     result = []  
4     def backtrack(start, remaining, combination):  
5         if remaining == 0:  
6             result.append(combination[:])  
7             return  
8         for i in range(start, len(candidates)):  
9             if i > start and candidates[i] == candidates[i - 1]:  
10                 continue  
11             if candidates[i] > remaining:  
12                 break  
13             combination.append(candidates[i])  
14             backtrack(  
15                 i + 1,  
16                 remaining - candidates[i],  
17                 combination  
18             )  
19             combination.pop()  
20     backtrack(0, target, [])  
21     return result  
22 print(combinationSum2(  
23     [10, 1, 2, 7, 6, 1, 5], 8))
```

Output:

Output
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

Time Complexity – $O(2^N)$ | Space Complexity – $O(N)$

Q9. Generate All Permutations

Aim:

To generate all possible permutations of a given array using Backtracking.

Algorithm:

Step 1: Start with an empty permutation.

Step 2: Select an unused number.

Step 3: Add it to the current permutation.

Step 4: Recursively generate the remaining permutation.

Step 5: Remove the number using Backtracking.

Step 6: Store the permutation when all numbers are used.

Python Code:

```
main.py Visualize Run
1 def permute(nums):
2     result = []
3     def backtrack(path, used):
4         if len(path) == len(nums):
5             result.append(path[:])
6             return
7         for i in range(len(nums)):
8             if not used[i]:
9                 used[i] = True
10                path.append(nums[i])
11                backtrack(path, used)
12                path.pop()
13                used[i] = False
14        backtrack([], [False] * len(nums))
15    return result
16 print(permute([1, 2, 3]))
```

Output:

```
Output
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

Time Complexity – $O(N \times N!)$ | Space Complexity – $O(N)$

Q10. Unique Permutations

Aim:

To generate all unique permutations of an array containing duplicate elements.

Algorithm:

Step 1: Sort the array.

Step 2: Use Backtracking to generate permutations.

Step 3: Skip duplicate elements at the same recursion level.

Step 4: Store only unique permutations.

Python Code:

main.py Visualize Run

```
1 def permuteUnique(nums):
2     nums.sort()
3     result = []
4     def backtrack(path, used):
5         if len(path) == len(nums):
6             result.append(path[:])
7             return
8         for i in range(len(nums)):
9             if used[i]:
10                continue
11             if (i > 0 and
12                 nums[i] == nums[i - 1] and
13                 not used[i - 1]):
14                continue
15             used[i] = True
16             path.append(nums[i])
17             backtrack(path, used)
18             path.pop()
19             used[i] = False
20     backtrack([], [False] * len(nums))
21     return result
22 print(permuteUnique([1, 1, 2]))
```

Output:

Output

```
[[1, 1, 2], [1, 2, 1], [2, 1, 1]]
```

Time Complexity – $O(N \times N!)$ | Space Complexity – $O(N)$

Q11. Graph Coloring

Aim:

To implement the Graph Coloring technique using Backtracking and find a valid coloring using the minimum number of colors.

Algorithm:

Step 1: Create the graph using the given edges.

Step 2: Start coloring vertices one by one.

Step 3: Check whether a color can be safely assigned.

Step 4: Ensure adjacent vertices do not have the same color.

Step 5: If a color is valid, assign it.

Step 6: If no color is possible, backtrack.

Python Code:

```
main.py Visualize Run
1 def graphColoring(n, edges, k):
2     graph = [[] for _ in range(n)]
3     for u, v in edges:
4         graph[u].append(v)
5         graph[v].append(u)
6     colors = [0] * n
7     def is_safe(vertex, color):
8         for neighbor in graph[vertex]:
9             if colors[neighbor] == color:
10                return False
11        return True
12    def solve(vertex):
13        if vertex == n:
14            return True
15        for color in range(1, k + 1):
16            if is_safe(vertex, color):
17                colors[vertex] = color
18                if solve(vertex + 1):
19                    return True
20                colors[vertex] = 0
21        return False
22    if solve(0):
23        return colors
24    return None
25 edges = [
26     (0, 1), (1, 2),
27     (2, 3), (3, 0),
28     (0, 2)
29 ]
30 print("Vertex colors:",
31       graphColoring(4, edges, 3))
```

Output:

```
Output
Vertex colors: [1, 2, 3, 2]
```

Time Complexity – $O(M^V)$ | Space Complexity – $O(V)$

Q12. Hamiltonian Cycle

Aim:

To determine whether a Hamiltonian Cycle exists in an undirected graph using Backtracking.

Algorithm:

Step 1: Start from vertex 0.

Step 2: Add an unvisited adjacent vertex to the path.

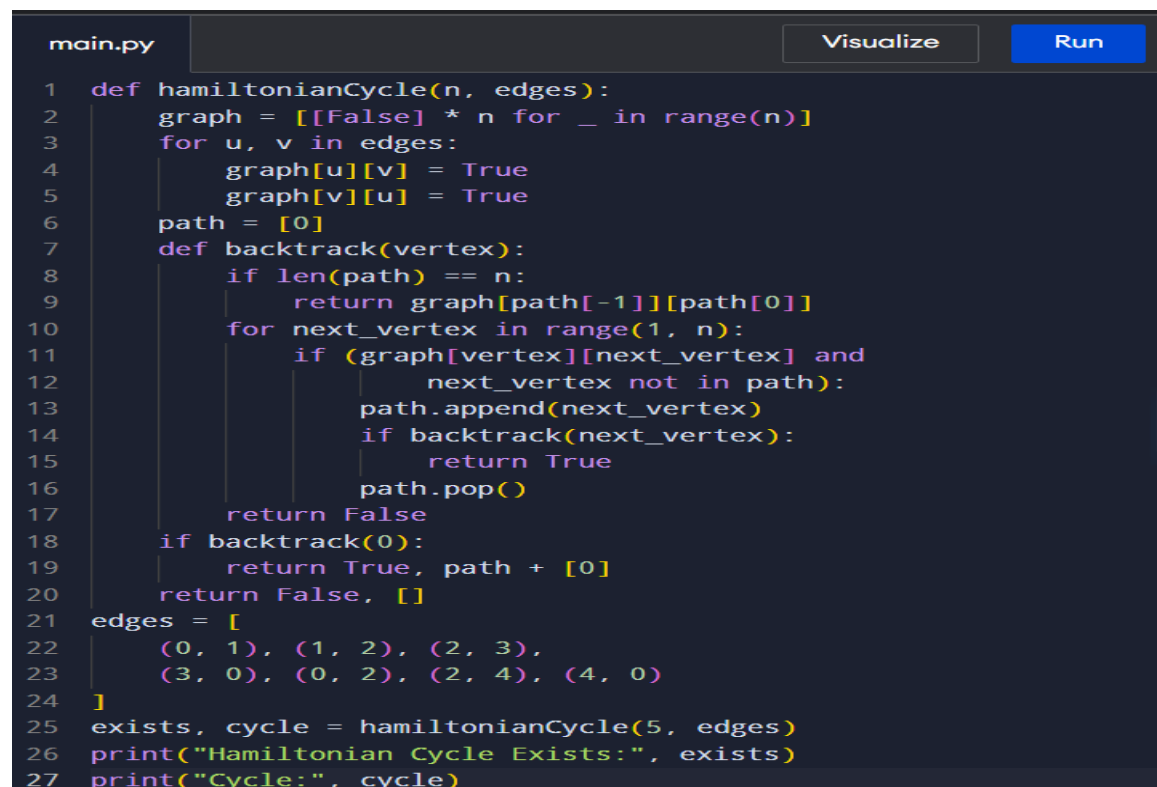
Step 3: Continue until all vertices are included.

Step 4: Check whether the last vertex connects back to the first vertex.

Step 5: If yes, a Hamiltonian Cycle exists.

Step 6: Otherwise, backtrack and try another path.

Python Code:



```
main.py Visualize Run
1 def hamiltonianCycle(n, edges):
2     graph = [[False] * n for _ in range(n)]
3     for u, v in edges:
4         graph[u][v] = True
5         graph[v][u] = True
6     path = [0]
7     def backtrack(vertex):
8         if len(path) == n:
9             return graph[path[-1]][path[0]]
10        for next_vertex in range(1, n):
11            if (graph[vertex][next_vertex] and
12                next_vertex not in path):
13                path.append(next_vertex)
14                if backtrack(next_vertex):
15                    return True
16                path.pop()
17        return False
18    if backtrack(0):
19        return True, path + [0]
20    return False, []
21 edges = [
22     (0, 1), (1, 2), (2, 3),
23     (3, 0), (0, 2), (2, 4), (4, 0)
24 ]
25 exists, cycle = hamiltonianCycle(5, edges)
26 print("Hamiltonian Cycle Exists:", exists)
27 print("Cycle:", cycle)
```

Output:

Output

Hamiltonian Cycle Exists: False
Cycle: []

Time Complexity – $O(V!)$ | Space Complexity – $O(V)$

Q14. Hamiltonian Cycle for Another Graph

Aim:

To find a Hamiltonian Cycle in the given undirected graph using Backtracking.

Algorithm:

Step 1: Start from a vertex.

Step 2: Visit each vertex exactly once.

Step 3: Check whether consecutive vertices have edges.

Step 4: After visiting all vertices, check whether the last vertex connects to the starting vertex.

Step 5: Use Backtracking when a path is not possible.

Python Code:

```
main.py Visualize Run
1 def hamiltonianCycle(n, edges):
2     for u, v in edges:
3         graph[u].append(v)
4         graph[v].append(u)
5     path = [0]
6     def solve(v):
7         if len(path) == n:
8             return 0 in graph[v]
9         for next_v in graph[v]:
10            if next_v not in path:
11                path.append(next_v)
12                if solve(next_v):
13                    return True
14                path.pop()
15            return False
16        if solve(0):
17            return True, path + [0]
18        return False, []
19 edges = [
20     (0, 1),
21     (1, 2),
22     (2, 3),
23     (3, 0),
24     (0, 2)
25 ]
26 exists, cycle = hamiltonianCycle(4, edges)
27 print("Hamiltonian Cycle Exists:", exists)
28 if exists:
29     print("Example cycle:", cycle)
```


Output:

Output
Hamiltonian Cycle Exists: True Example cycle: [0, 1, 2, 3, 0]

Time Complexity – $O(V!)$ | Space Complexity – $O(V)$

Q15. Generate All Subsets

Aim:

To generate all subsets of a given set using Backtracking.

Algorithm:

Step 1: Start with an empty subset.

Step 2: For every element, decide whether to include it.

Step 3: Recursively generate subsets.

Step 4: Store every generated subset.

Step 5: Return all subsets.

Python Code:

```
main.py Visualize Run
1 def subsets(nums):
2     result = []
3     def backtrack(index, current):
4         result.append(current[:])
5         for i in range(index, len(nums)):
6             current.append(nums[i])
7             backtrack(i + 1, current)
8             current.pop()
9     backtrack(0, [])
10    return result
11 A = [1, 2, 3]
12 print("Subsets:")
13 print(subsets(A))
```

Output:

Output

Subsets:

```
[[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
```

Time Complexity – $O(2^N)$ | Space Complexity – $O(N)$

Q16. Generate Subsets Containing Element 3

Aim:

To generate all subsets containing a specific element using Backtracking.

Algorithm:

Step 1: Generate all possible subsets.

Step 2: Check whether each subset contains the required element.

Step 3: Store subsets containing that element.

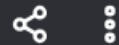
Step 4: Return the result.

Python Code:

```
main.py Visualize Run
1 def subsetsContaining(nums, x):
2     result = []
3     def backtrack(index, current):
4         if index == len(nums):
5             if x in current:
6                 result.append(current[:])
7             return
8         current.append(nums[index])
9         backtrack(index + 1, current)
10        current.pop()
11        backtrack(index + 1, current)
12    backtrack(0, [])
13    return result
14 E = [2, 3, 4, 5]
15 print("Subsets containing 3:")
16 print(subsetsContaining(E, 3))
```

Output:

Output



```
Subsets containing 3:  
[[2, 3, 4, 5], [2, 3, 4], [2, 3, 5], [2, 3], [3, 4, 5], [3, 4],  
[3, 5], [3]]
```

Time Complexity – $O(2^N)$ | Space Complexity – $O(N)$

Q17. Word Subsets

Aim:

To find all universal strings in words1 such that every string in words2 is a subset of them.

Algorithm:

Step 1: Count the frequency of characters in every word of words2.

Step 2: Find the maximum required frequency of every character.

Step 3: Check every word in words1.

Step 4: Verify whether it contains all required characters with sufficient frequency.

Step 5: Add valid words to the result.

Python Code:

```
main.py Visualize Run
2 def wordSubsets(words1, words2):
4     for word in words2:
6         for char in count:
7             required[char] = max(
8                 required[char],
9                 count[char]
10            )
11     result = []
12     for word in words1:
13         count = Counter(word)
14         valid = True
15         for char in required:
16             if count[char] < required[char]:
17                 valid = False
18                 break
19         if valid:
20             result.append(word)
21     return result
22 words1 = [
23     "amazon",
24     "apple",
25     "facebook",
26     "google",
27     "leetcode"
28 ]
29 words2 = ["e", "o"]
30 print(wordSubsets(words1, words2))
31 words2 = ["l", "e"]
32 print(wordSubsets(words1, words2))
```

Output:

```
Output
['facebook', 'google', 'leetcode']
['apple', 'google', 'leetcode']
```

Time Complexity – $O(N \times L)$ | Space Complexity – $O(1)$